E-Challan System — Data Structures Reference

This document lists every data structure in the `E-Challan-System` repository: the Mongoose schemas in `1/server/models/`, the mock data in `1/src/data/mockChallans.js`, and the other structured objects the app uses (API payloads, React context state, form models, browser storage keys, and third-party config objects). For each one it gives the fields, types, constraints, and purpose.

Summary

#	Structure	Location	Kind	Persisted?
1	`Challan`	`1/server/models/Challan.js`	Mongoose model	Yes (MongoDB `challans`)
2	`Vehicle`	`1/server/models/Vehicle.js`	Mongoose model	Yes (MongoDB `vehicles`)
3	`mockChallans`	`1/src/data/mockChallans.js`	Static JS array	No
4	Seed dataset	`1/server/routes/challanRoutes.js` (`POST /seed`)	Static JS array	Written to MongoDB on demand
5	API request/response shapes	`1/server/routes/*.js`	JSON payloads	No
6	Admin challan form	`1/src/pages/AdminAddRecord.jsx`	React state	Sent to API
7	Legacy user record form	`1/src/pages/AddRecord.jsx`	React state	No (see notes)
8	`RecordsContext` value and stats	`1/src/contexts/RecordsContext.jsx`	React context	No
9	`AuthContext` value	`1/src/contexts/AuthContext.jsx`	React context	Firebase Auth (localStorage)
10	Session storage keys	Several pages	`sessionStorage`	Browser tab only
11	Firebase config	`1/src/firebase.jsx`	Config object	No
12	Cloudinary config and upload options	`1/src/utils/cloudinary.js`	Config object	No

There are only two Mongoose models. No other model files exist anywhere in the repo, and Firestore and Firebase Storage are initialized but never used to read or write data.

---

1. `Challan` (Mongoose model)

File: `1/server/models/Challan.js`
Collection: `challans` (Mongoose pluralizes the model name `Challan`)
Purpose: One traffic violation (fine) issued against a licence plate, plus its payment status. This is the main business entity: users look up challans by vehicle number, admins create them, and the frontend computes totals and analytics from them.

Fields

Field	Type	Required	Unique	Other constraints	Description
`_id`	`ObjectId`	auto	yes	Added by Mongoose	Internal MongoDB primary key. The API does not use it to look records up.
`id`	`String`	yes	yes (unique index)	—	Human-readable challan number, e.g. `CHLN001`. The business key, used by `PUT /api/challans/:id`.
`date`	`String`	yes	no	No format check	Date of the violation. By convention `YYYY-MM-DD` (e.g. `2024-10-15`). Stored as a string, not a `Date`.
`time`	`String`	yes	no	No format check	Time of the violation. By convention 24-hour `HH:mm` (e.g. `14:30`).
`type`	`String`	yes	no	Free text, no enum	Violation category, e.g. `Speeding`, `Signal Jump`, `No Parking`, `Helmet Violation`.
`location`	`String`	yes	no	—	Where the violation happened, e.g. `FC Road, Pune`.
`licensePlate`	`String`	yes	no	Not normalized by the schema	Plate of the offending vehicle, e.g. `MH12AB1234`. Logically joins to `Vehicle.vehicleNumber` (see Relationships).
`amount`	`Number`	yes	no	No `min`; decimals allowed	Fine amount. The seed data uses INR (500, 1000).
`status`	`String`	yes	no	`enum: ["Pending", "Paid"]`	Payment state. `POST /api/challans` defaults it to `"Pending"` when it is missing.
`__v`	`Number`	auto	no	Added by Mongoose	Document version key.

Constraints and behaviour

- **No timestamps:** the schema does not set `{ timestamps: true }`, so there are no `createdAt` / `updatedAt` fields.
- **State machine:** `Pending → Paid`. Nothing in the schema stops a change from `Paid` back to `Pending`.
- **Validation on update:** `PUT /api/challans/:id` calls `findOneAndUpdate` without `runValidators: true`. Updates therefore skip the `required` and `enum` checks, so an arbitrary `status` value can be written. The route deletes `updates.id` so the business key can't be changed.
- **Lookup by plate:** `GET /api/challans?vehicleNumber=...` loads *all* challans and filters them in memory. Both sides are compared after removing whitespace and upper-casing, so `mh12 ab 1234` matches `MH12AB1234`.

Example document

```
{
  "_id": "665f1c2e9b1e8a0012345678",
  "id": "CHLN001",
  "date": "2024-10-15",
  "time": "14:30",
  "type": "Speeding",
  "location": "Mumbai-Pune Expressway",
  "licensePlate": "MH12AB1234",
  "amount": 1000,
  "status": "Pending",
  "__v": 0
}
```

---

2. `Vehicle` (Mongoose model)

File: `1/server/models/Vehicle.js`
Collection: `vehicles`
Purpose: Records which user (by email) owns a vehicle number. The server uses it for *ownership enforcement*: a user can only view challans for a vehicle registered to their email, or for a vehicle nobody has registered yet.

Fields

Field	Type	Required	Unique	Other constraints	Description
`_id`	`ObjectId`	auto	yes	Added by Mongoose	Internal primary key.
`vehicleNumber`	`String`	yes	yes (unique index)	`uppercase: true` (Mongoose setter)	Normalized plate number. The routes also remove all whitespace before saving or querying, so stored values look like `MH12AB1234`.
`ownerEmail`	`String`	yes	no	No format or lowercase check	Email of the Firebase-authenticated owner. Compared with exact, case-sensitive string equality.
`createdAt`	`Date`	no	no	`default: Date.now`	When the vehicle was registered.
`__v`	`Number`	auto	no	Added by Mongoose	Version key.

Constraints and behaviour

- **One owner per vehicle:** the unique index on `vehicleNumber` and the `POST /api/vehicles/register` logic ensure each plate has at most one owner. One email can own many vehicles.
- **Registration outcomes:**
- New plate → `201` with the created document.
- Already registered to the same email → `200` with `{ message, vehicle }`.
- Registered to another email → `409` with `{ message }`.

- **Ownership check on challan lookup:** if `GET /api/challans` gets both `vehicleNumber` and `userEmail`, and the vehicle is registered to a different email, it returns `403` with `{ code: "VEHICLE_OWNED_BY_OTHER", message }`. Unregistered vehicles are not blocked.

Example document

```
{
  "_id": "665f1d0a9b1e8a0012345679",
  "vehicleNumber": "MH12AB1234",
  "ownerEmail": "user@example.com",
  "createdAt": "2024-12-10T08:15:30.000Z",
  "__v": 0
}
```

Relationships

```
Vehicle (1) ——— owns ———< (0..n) Challan
  vehicleNumber  ≈  normalize(Challan.licensePlate)
  where normalize(s) = s.replace(/\s/g, "").toUpperCase()
```

The link is **logical only**. There is no `ref`, foreign key, or index on `Challan.licensePlate`. A challan can exist for a plate that has no `Vehicle` document.

---

3. `mockChallans` (static mock data)

File: `1/src/data/mockChallans.js`
Export: `export const mockChallans: Challan[]` (named export)
Purpose: Sample challan records for frontend development and demos. Each element has the same shape as the `Challan` model (without `_id` / `__v`).

Usage: nothing in the codebase currently imports `mockChallans`. The same five records are hard-coded again in the server's `POST /api/challans/seed` route (section 4), which is how sample data actually gets into MongoDB.

Element shape

Field	Type	Example	Notes
`id`	`string`	`"CHLN001"`	Format `CHLN` + 3-digit number
`date`	`string`	`"2024-10-15"`	`YYYY-MM-DD`
`time`	`string`	`"14:30"`	`HH:mm`, 24-hour
`type`	`string`	`"Speeding"`	Violation category
`location`	`string`	`"Mumbai-Pune Expressway"`	Free text
`licensePlate`	`string`	`"MH12AB1234"`	Uppercase, no spaces
`amount`	`number`	`1000`	INR
`status`	`"Pending" | "Paid"`	`"Pending"`	Payment state

Contents

id	date	time	type	location	licensePlate	amount	status
CHLN001	2024-10-15	14:30	Speeding	Mumbai-Pune Expressway	MH12AB1234	1000	Pending
CHLN002	2024-11-02	09:15	Signal Jump	Deccan Gymkhana, Pune	MH12AB1234	500	Pending
CHLN003	2024-09-20	18:45	No Parking	FC Road, Pune	MH12AB1234	500	Paid
CHLN004	2024-12-01	10:00	Helmet Violation	Shivaji Nagar	MH14XY9876	500	Pending
CHLN005	2024-12-05	11:30	Speeding	Baner Road	MH14XY9876	1000	Paid

Totals: 5 challans across 2 vehicles. 3 Pending (₹2,000), 2 Paid (₹1,500).

---

4. Seed dataset (`POST /api/challans/seed`)

File: `1/server/routes/challanRoutes.js`
Purpose: Resets the `challans` collection to a known state. The handler runs `Challan.deleteMany({})` and then `Challan.insertMany(sampleData)`.

`sampleData` is a local array with exactly the same five records as `mockChallans` (section 3). Because the two copies are separate, a change to one does not update the other. The route needs no authentication and deletes every existing challan.

---

5. API request and response shapes

Base path is set in `1/server/server.js`: `/api/challans` → `challanRoutes`, `/api/vehicles` → `vehicleRoutes`. Every error response has the shape `{ message: string }` unless the table says otherwise.

Challan endpoints

Method and path	Request	Success response	Error responses
`GET /api/challans`	Query (optional): `vehicleNumber: string`, `userEmail: string`	`200` `Challan[]`	`403` `{ code: "VEHICLE_OWNED_BY_OTHER", message }`; `500`
`POST /api/challans`	Body: `{ id, date, time, type, location, licensePlate, amount, status? }`	`201` `Challan`	`400` (validation / duplicate `id`)
`PUT /api/challans/:id`	Path: business `id`. Body: partial `Challan` (`id` is ignored)	`200` updated `Challan`	`404` `{ message: "Challan not found" }`; `400`
`POST /api/challans/seed`	none	`200` `{ message: "Database seeded successfully" }`	`500`

Vehicle endpoints

Method and path	Request	Success response	Error responses
`POST /api/vehicles/register`	Body: `{ vehicleNumber: string, email: string }`	`201 Vehicle`, or `200 { message: "Vehicle already registered to you", vehicle: Vehicle }`	`400 { message: "Missing fields" }`; `409`; `500`
`GET /api/vehicles/my-vehicles`	Query: `email: string` (required)	`200 Vehicle[]`	`400 { message: "Email required" }`; `500`
`GET /api/vehicles/status/:vehicleNumber`	Path: plate. Query (optional): `email`	`200 { status: "unregistered" | "owned_by_self" | "owned_by_other" }`	`500`

`VehicleStatus` enum

Value	Meaning
`unregistered`	No `Vehicle` document for this plate
`owned_by_self`	Registered, and `ownerEmail` equals the supplied `email`
`owned_by_other`	Registered to someone else, or no `email` was supplied

---

6. Admin challan form (`AdminAddRecord`)

File: `1/src/pages/AdminAddRecord.jsx`
Purpose: React state behind the admin "Add New Record" form. When submitted, it is sent to `POST /api/challans` through `challanService.createChallan`.

Field	Type (in state)	Initial value	Notes
`id`	`string`	`` `CHLN${Math.floor(Math.random() * 10000)}` ``	Read-only input. Random 0–9999, so it can collide with an existing `id`, which the unique index would reject.
`date`	`string`	`""`	Date input
`time`	`string`	`""`	Time input
`type`	`string`	`"Speeding"`	Violation type select
`location`	`string`	`""`	
`licensePlate`	`string`	`""`	
`amount`	`string`	`"500"`	Converted with `Number()` before sending
`status`	`string`	`"Pending"`	

The submitted object matches the `Challan` schema exactly.

---

7. Legacy user record form (`AddRecord`)

File: `1/src/pages/AddRecord.jsx`
Purpose: An older form for creating violation records. It builds a record whose shape is **different** from the `Challan` model.

Field	Type	Notes
`violationType`	`string`	One of `Speeding`, `Red Light`, `Parking Violation`, `No Seat Belt`, `Illegal U-Turn`, `Stop Sign Violation`, `Wrong Lane`, `Other`. Corresponds to `Challan.type`.
`date`	`string`	
`time`	`string`	
`location`	`string`	
`licensePlate`	`string`	Upper-cased
`amount`	`number`	`parseFloat`
`status`	`"Pending"`	Always Pending
`dueDate`	`string`	`YYYY-MM-DD`; defaults to today + 30 days. **Not in the `Challan` schema.**
`description`	`string`	**Not in the `Challan` schema.**

The record has no `id`. It is passed to `addRecord(...)` from `RecordsContext`, but `RecordsContext` does not provide `addRecord` (section 8), so this flow does not save anything. `ViewAllRecords.jsx` also reads `record.violationType`. `Payment.jsx` handles both shapes (`record.type || record.violationType`).

---

8. `RecordsContext` value and stats

File: `1/src/contexts/RecordsContext.jsx`
Purpose: Client-side store for the challans currently loaded for one vehicle.

Context value

Member	Type	Description
`records`	`Challan[]`	Challans loaded by the latest `fetchRecordsByVehicle` call
`loading`	`boolean`	A fetch is in progress
`error`	`string | null`	`"Failed to fetch records"` when a fetch fails
`fetchRecordsByVehicle(vehicleNumber)`	`async (string) => void`	Calls `GET /api/challans?vehicleNumber=...`
`clearRecords()`	`() => void`	Empties `records` (for example on logout)
`getStats()`	`() => RecordStats`	Totals computed from the loaded records (below)
`updateRecord(id, updates)`	`async (string, Partial<Challan>) => void`	Calls `challanService.updateChallan`, then merges `updates` into local state

`RecordStats`

Field	Type	Calculation
`totalViolations`	`number`	`records.length`
`pendingPayments`	`number`	Count where `status === "Pending"`
`paidFines`	`number`	Count where `status === "Paid"`
`totalAmount`	`number`	Sum of `amount`
`pendingAmount`	`number`	Sum of `amount` where `status === "Pending"`

---

9. `AuthContext` value

File: `1/src/contexts/AuthContext.jsx` (used through `1/src/hooks/useAuth.jsx`)
Purpose: Exposes the Firebase Authentication session. The session persists in `localStorage` (`browserLocalPersistence`).

Member	Type	Description
`user`	Firebase `User | null`	Signed-in user. `user.email` is used as `ownerEmail` / `userEmail` in API calls
`loading`	`boolean`	`true` until the first auth state is known
`error`	`Error | null`	Auth listener error
`signIn(email, password)`	`Promise<UserCredential>`	Email/password sign-in
`signUp(email, password)`	`Promise<UserCredential>`	Email/password registration
`signOut()` / `logout()`	`Promise<void>`	Sign out (two names for the same function)
`googleSignIn()`	`Promise<UserCredential>`	Google sign-in popup

There is no user collection in MongoDB. User identity lives only in Firebase Auth.

---

10. Browser `sessionStorage` keys

Key	Value type	Set in	Read in	Purpose
`isAdmin`	`"true"` (string)	`AdminLogin.jsx`	`AdminDashboard.jsx`, `AdminAddRecord.jsx`	Client-side flag that lets the browser show admin pages. Removed on admin logout. The server never checks it.
`searchVehicle`	`string` (plate)	`LandingPage.jsx`, `UserDashboard.jsx`	`UserDashboard.jsx`	Keeps the last searched vehicle number across navigation and reloads

---

11. Firebase configuration object

File: `1/src/firebase.jsx`

Field	Source env var
`apiKey`	`REACT_APP_FIREBASE_API_KEY`
`authDomain`	`REACT_APP_FIREBASE_AUTH_DOMAIN`
`projectId`	`REACT_APP_FIREBASE_PROJECT_ID`
`storageBucket`	`REACT_APP_FIREBASE_STORAGE_BUCKET`
`messagingSenderId`	`REACT_APP_FIREBASE_MESSAGING_SENDER_ID`
`appId`	`REACT_APP_FIREBASE_APP_ID`

The file exports `auth`, `db` (Firestore), `storage` (Cloud Storage), and the default `app`. Only `auth` is used; no Firestore collections are defined or read.

---

12. Cloudinary configuration and upload options

Files: `1/src/utils/cloudinary.js`, `1/src/config/cloudinary.jsx`

`uploadToCloudinary(file, options)` expects a `cloudinaryConfig` object and a `CLOUDINARY_UPLOAD_URL` string:

Field	Type	Source
`cloudinaryConfig.cloudName`	`string`	`REACT_APP_CLOUDINARY_CLOUD_NAME`
`cloudinaryConfig.uploadPreset`	`string`	`REACT_APP_CLOUDINARY_UPLOAD_PRESET` (unsigned preset)
`options`	`Record<string, string>`	Optional extra Cloudinary parameters (for example `folder`, `tags`) appended to the `FormData`

The function returns Cloudinary's upload response as JSON. Note that `1/src/config/cloudinary.jsx` imports `cloudinaryConfig` and `CLOUDINARY_UPLOAD_URL` from `./cloudinary`, which is itself. It contains a copy of the upload function instead of defining those exports, so they are never actually defined.

Observations and inconsistencies

These came up while documenting the structures. They are listed for reference only; this change does not fix them.

1. **Two record shapes.** `Challan` uses `type` and has no `dueDate` / `description`. `AddRecord.jsx` and `ViewAllRecords.jsx` use `violationType`, `dueDate`, and `description`.
2. **Missing context and service methods.** `RecordsContext` calls `challanService.updateChallan`, which `challanService.js` does not define, so `updateRecord` (used by "Pay Now") throws, catches the error, and changes nothing. `addRecord` and `deleteRecord` are used by pages but not provided by the context.
3. **No validation on update.** `PUT /api/challans/:id` doesn't pass `runValidators: true`, so `status` can be set outside `["Pending", "Paid"]`.

4. **Duplicated sample data.** `mockChallans.js` and the `/seed` route have separate copies of the same five records, and `mockChallans` isn't imported anywhere.
5. **Weak typing of date and time.** `date` and `time` are free-form strings with no format check. Sorting and analytics depend on the `YYYY-MM-DD` convention being followed.
6. **Random challan IDs.** Admin-generated IDs (`CHLN` + 0–9999) can collide with existing IDs.
7. **Plate normalization only in routes.** `Vehicle.vehicleNumber` is upper-cased by the schema but has whitespace removed only in route code. `Challan.licensePlate` is not normalized at all; lookups normalize it at query time.
8. **Case-sensitive email matching.** `ownerEmail` is compared case-sensitively, so `User@x.com` and `user@x.com` count as different owners.
9. **Client-only admin flag.** Admin access depends only on the `isAdmin` key in `sessionStorage`; the challan write endpoints have no server-side authorization.